Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

[Project Report]

Project Title:
Cumulative Grade Point Average Calculator

Submitted by:
Md. Shoibe Hossain Rifat
Student ID: 11240321705
Email: shoiberifat797@gmail.com

Course Title: Object Oriented Programming ll
Course Code: CSE 2110
Section: 3A

Submitted to:
Shovon Mandal [SM]
Lecturer,
Department of Computer Science and Engineering
Northern University of Business and Technology Khulna

Date of Submission:
27th March 2026

Table of Contents

Cumulative Grade Point Average Calculator

Abstract:

The Cumulative Grade Point Average Calculator is a Java Swing application that calculates CGPA for courses efficiently and accurately. Calculating CGPA manually is tedious and prone to errors, especially if more subjects or courses are involved. This application automates that process by accepting some information such as (subject name, grade point, credit hours) and applying a weighted-average formula (total quality points divided by total credits) to calculate CGPA. Developed using Java, it features multiple buttons: **Add Subject** (take entries), **Remove** (clears all entry and text-area), and **Calculate CGPA** (performs calculation). User provided input is validated by checking, if the given grade point is within the valid range and credit hours is a non-negative number or non-zero. Testing results show that the application accurately calculates CGPA and handles invalid inputs as expected.

Introduction:

1. Background:

A standard measure of a students overall academic performance is their CGPA (Cumulative Grade Point Average) commonly used in all academic institutions. It is a combination of both grades and credit hours. Specifically, saying each course's merit points are its grade (typically on a scale of 0–4.0) multiplied by its credit hours, and CGPA is the sum of all these merit points divided by the sum of all credits. In short, CGPA gives an overview of a students overall performance over multiple courses over multiple semesters.

2. Problem Statement:

Calculating CGPA manually is time consuming and prone to error, especially when students have many courses in a semester or over multiple semesters. Students have to multiply each grade by its credits and sum their values, they have to do this multiple times for multiple courses. So, even a small arithmetic mistake can lead to incorrect CGPA results.

3. Objectives:

The main objectives of this project are:

1. To make an application in Java that calculates CGPA based on user defined data.
2. To use Java Swing to build an interactive user interface for entering data and displaying the result.
3. To calculate CGPA using an academic standard weighted formula:
   CGPA = (Grade Point Value * Credit Hours)/ Total Credit Hours

4. To ensure users can add any number of courses at runtime.
5. To ensure data validation by checking inputs for correctness (numeric values, valid grade range, non-zero or non-negative credits).
6. To solve tedious manual CGPA calculation by automating the whole process.

4. Scope:

This Java Swing desktop application calculates CGPA on a (4.0 scale) using integer credit hours. Users can manually input each course's name, grade point, and credits. No external data sources are to be used. It calculates and displays the CGPA on command in a session only environment, without databases or preserving storage. The focus is strictly on CGPA calculation.

Literature Review:

1. Student Grade Calculator Using Java Swing.
This project application will primarily be based on “Student Grade Calculator using Java Swing“ tutorial from GeeksforGeeks website. It shows basic implementation of swing components with a fixed text field for marks, buttons used to calculate percentage and if-else-if ladder for grade letter logic based on percentage. But the calculation logic is flawed for cumulative grade calculation as well as their hard coded text fields for subject’s marks [1].

2. Java Swing Bangla Tutorial.
Anisul Islam’s “Java Swing Bangla Tutorial” playlist is a beginner friendly tutorial for Java Swing. It demonstrates Java Swing’s components and their use cases which is a prerequisite for any kind of GUI based application. This playlist is a full on guide on how to use java swing and implement every basic components as a beginner. From importing swing components to adding action listener, changing font, to set the frames location, size and more [2].

3. Calculation Logic.
Ohio State University’s resources page provides a set of clean academic standard cumulative grade point calculation logic for 4.0 scale [3].

What I tend to do differently from the “Student Grade Calculator using Java Swing” tutorial, from GeeksforGeeks [1]. Is to add a new cumulative grade point average calculation logic, and an university standard grade point scale and credit hours per course [3]. And instead of hard coded fields, the user will be able to add many subjects as needed [1]. And the positioning of the layout will be manually set [2].

Methodology:

1. Approach:

The Cumulative Grade Point Average Calculator will be developed using Java programming language and Java Swing. Its development follows a straightforward approach combining GUI design, algorithmic design, mathematical constructs, and object oriented programming principles. The approach involves:

1. Class based structures for modularity through Object Oriented Programming.
2. Action Listener for event driven programming.
3. Input validation for ensuring correct data entries.
4. Modular design to keep input, data processing and output separate.
5. String slicing, parsing and numerical conversion for calculation.

2. Algorithms:

The main technique will be the weighted average algorithm that is used for cumulative grade point average calculation.

Steps:

1. Start the application.
2. Takes the user defined data input of subject name, grade(0-4), and credit hours through the text field.
3. Validates input data for any errors by checking for empty fields, and as it parses grade and credits as numbers it checks if grade is in between (0-4.0) and credit hours are non-zero and non-negative number.
4. If input is valid, then stores the given data temporarily displaying them on the text area.
5. For other subjects the process is the same as in 2,3 and 4.
6. When the Calculate CGPA button is clicked, it calculates the quality point for each subject. And calculates the total quality points along with the total credit hours.
7. If the display area is empty, then it shows an error message on the window.
8. Else it calculates the cumulative grade point average by dividing the (Total quality points) by (Total credit hours). If no courses were entered (total credits = 0), an error message is shown instead of dividing by zero.
9. Displays the calculated cumulative grade point average.
10. Ends process.

This algorithmic design ensures a proper weighted calculation process.

3. Flowchart:

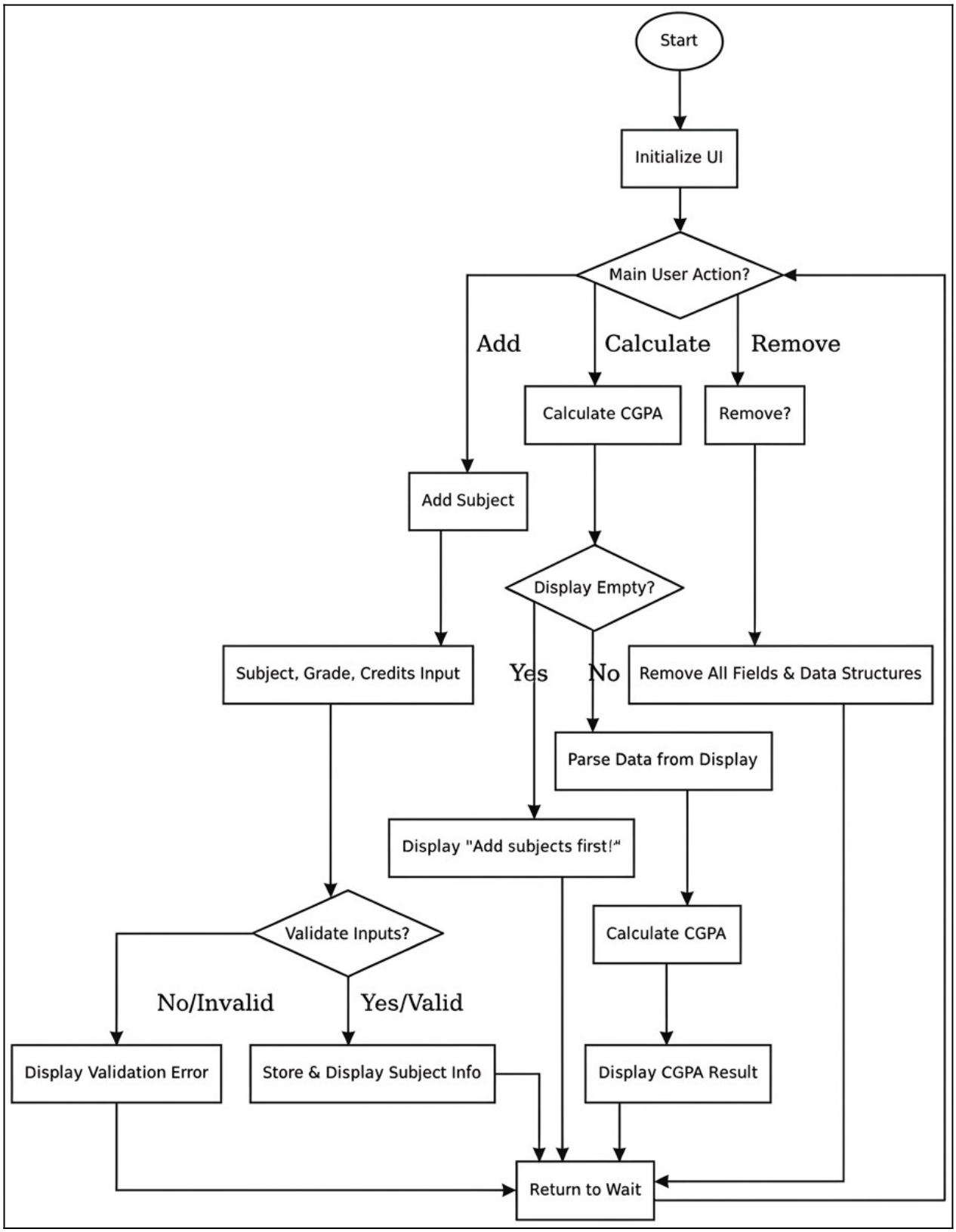

Figure 01: Flowchart of the Cumulative Grade Point Average Calculator System Workflow

4. Tools and Technologies:

1. **Operating System:** Linux (Cinnamon)
2. **Programming Language:** Java (JDK 21.0.1)
3. **Development environment:** Apache NetBeans IDE
4. **GUI Framework:** Java Swing

5. Experimental Design (Testing):

The application is tested using different test cases like:

1. Valid and Invalid data entries.
2. Empty text fields to check for error messages, as well as non-numeric entries in the credit and grade section shall flag an error.
3. Credit is entered as zero and negative values and grade out of range to check the validation logic.
4. Multiple subjects entries are added as well as clearing all entries and verifying the calculated CGPA.

These test scenarios will ensure the accuracy and stable workflow of this application.

Implementation:

1. System Architecture:

The implementation uses a modular structure:

1. **Front-end:** Encompasses all Java Swing components that are necessary to handle all user interaction. It takes input (subject, grade, credits) and displays output (list of courses and total CGPA).
2. **Logic Layer:** Contains the core logic flow. Event driven handlers (Action-listeners attached to buttons) validate inputs and performs necessary actions as programmed to do. When **Add Subject** is clicked, the listener validates text fields (non-empty, numeric, within valid range) and adds a new entry to the list. Clicking **Calculate CGPA** traverses all subject data entries and calculate total CGPA and displays the result.

3. **Control Flow:** As the user gives entries, **Add subject** button passes data for validation then the validated data is displayed in the **JTextArea** and the entry fields are cleared for new entries. **Calculate CGPA** button, slices then parses credit hours and grades from the JTextArea and calculates CGPA. By direct text area based storage system this application ensures that the view model synchronizes without separate data structures; so users can dynamically enter data. It Scales fine upto 20 to 30 subjects.

2. Components:

1. User Interface (UI)

1. **CGPACalculator.java**: Handles all the data entry fields and CGPA calculation logic and holds the main method, JFrame title, close operation, and size.
2. **JTextField** (tf1, tf2, tf3): These are input fields for subject name, grade, and credit hours.
3. **JTextArea** (display): It shows subject entries with grade and credit hours.
4. **JTextField** (tf4): It is the output field that shows the calculated CGPA.
5. **JButton** (bt1, bt2, bt3): These are buttons for adding, calculating, and clearing entries.

2. Data Validation

1. **Input Validation:** Checks if subject field empty, grade entered is within the valid range (0-4.0), and credits are positive and non-zero integers.
2. **Subject Entry:** Allows multiple subject entries with live display updates.

3. Core Logic

1. **Calculation Logic**: Applies an university standard weighted average formula to calculate CGPA from grade & credit inputs.
2. **Error Feedback:** Displays an error messages for invalid inputs using dialog boxes (showMessageDialog).
3. **Sessional Processing:** All operations are done in memory without saving data after closing the application.

3. Code Excerpts:

1. **Entering Subject's name, grade, and credit hours with entry validation logic.**

```
bt1.addActionListener(new ActionListener()
{
    @Override

    public void actionPerformed(ActionEvent e)
    {
       String name = tf1.getText();
       String grade = tf2.getText();
       String credit = tf3.getText();

       if(!name.isEmpty() && !grade.isEmpty() && !credit.isEmpty())
       {
           try
           {
               double Grade = Double.parseDouble(grade);
               int Credit = Integer.parseInt(credit);

               if(Grade<0 || Grade>4)
               {
                   JOptionPane.showMessageDialog(null, "Grade must be from 0 to 4");
                   return;
               }
               if(Credit <= 0)
               {
                   JOptionPane.showMessageDialog(null, "Credit cant be zero or negative");
                   return;
               }

               display.append("SUBJECT : "+name+" | GRADE: "+Grade+" | CREDITS: "+Credit+ "\n");

               tf1.setText("");
               tf2.setText("");// clears texfield on every entry
               tf3.setText("");

           }
           catch(NumberFormatException ex)
           {
             JOptionPane.showMessageDialog(null, "Credit and Grade must be numbers");
           }
       }

    }

});
```

2. Removing all entries.

```
bt2.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        tf1.setText("");
        tf2.setText("");
        tf3.setText("");
        tf4.setText("");
        display.setText("");

    }
});
```

3. Calculation logic with error handling.

```
bt3.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        String allsubs = display.getText().trim();
        //if display is empty
        if(allsubs.isEmpty())
        {
            JOptionPane.showMessageDialog(null, "Add some info first!");
            return;
        }
        String[] slines = allsubs.split("\n"); // splitting into lines
        double sumGradeCredits = 0;
        int totalCredits = 0;
        for(String lines : slines)
        {
            if(!lines.isEmpty())
            {
                try
                {
                    int StartGrade = lines.indexOf("GRADE: ")+7;
                    int EndGrade = lines.indexOf(" |", StartGrade);
                    String gradeStart = lines.substring(StartGrade, EndGrade);
                    double grade = Double.parseDouble(gradeStart);

                    int creditStart = lines.indexOf("CREDITS: ")+9;
                    String creditStri = lines.substring(creditStart);
                    int credits = Integer.parseInt(creditStri);

                    sumGradeCredits += grade * credits;
                    totalCredits += credits;
```

```
                }
                catch(Exception ex)
                {
                    JOptionPane.showMessageDialog(null, "Calculation Error");
                    return;
                }

            }
        }
        if(totalCredits > 0)
        {
            double cgpa = sumGradeCredits / totalCredits;
            tf4.setText(String.format("%.4f", cgpa));
        }
    }
});
```

Results and Analysis:

1. Results:

The CGPA Calculator was tested using multiple subjects with different grades and credit hours to check correctness of the resultant CGPA and edge case handling.

Table demonstrating multiple case results:

Test Case	Input (Grades & Credits)	Expected Outcome (CGPA)	Result
01	(4.0 , 3)	4.00	Pass
02	(3.0 , 3), (4.0 , 4)	3.57	Pass
03	(2.5 , 2), (3.5 , 3), (3.0 , 4)	3.22	Pass
04	("A" , 3), (4.0 , 4)	Error (non-numeric)	Handled
05	(3.0 , 0), (4.0 , 4)	Error (0 credit)	Handled

2. Analysis:

Test cases 1 to 3 (which are valid) are used to calculate results(CGPA) in the application, and they are verified by manual calculations. For Invalid, Test cases 4 & 5 they were handled by the application, which showed an error dialog instead of continuing. The performance is almost instant for small data sizes. The output CGPA is easy to verify and it is formatted to four decimal places for readability.

Overall, the results confirms the applications stability, and the weighted average formula operates correctly.

3. Snippet:

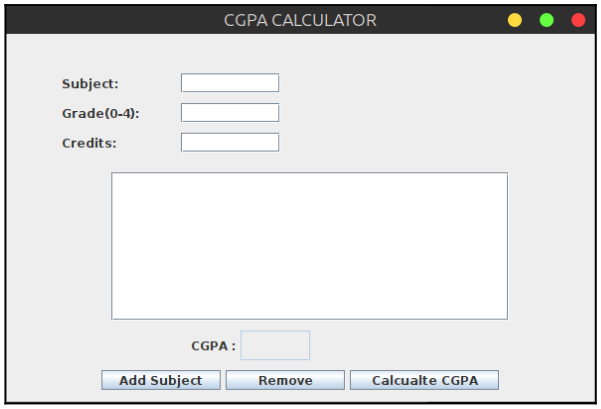

Fig 02: Main Window

Description: Running the program provides the interface where the user interacts.

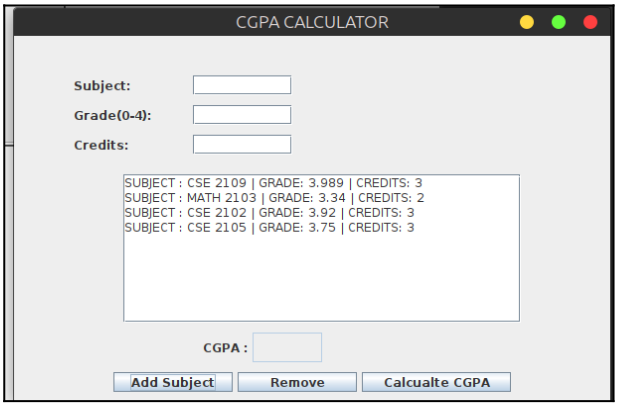

Fig 03: Course Added using Add Subject

Description: Added multiple subject entries using the “Add Subject” button.

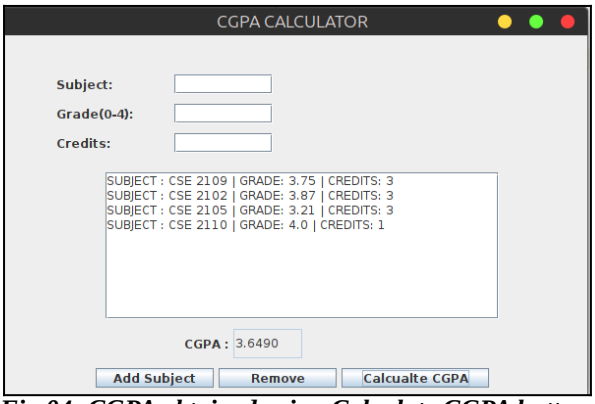

Fig 04: CGPA obtained using Calculate CGPA button

Description: Slicing and parsing the grade and credits the CGPA is obtained by using the Calculate CGPA button.

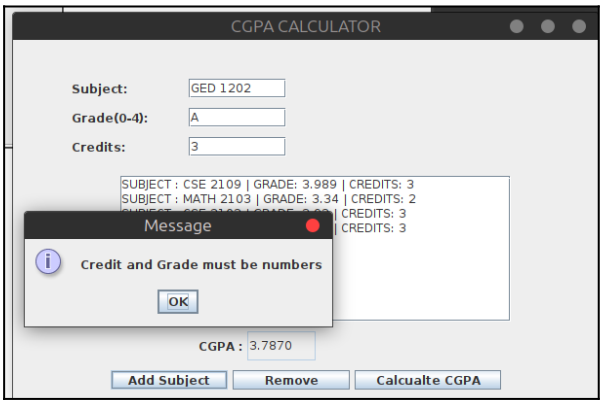

Fig 05: Handling Invalid Input

Description: Passed grade value as a character in which the invalid input is handled

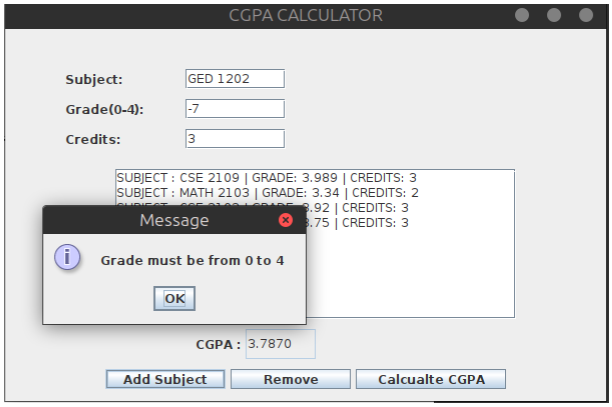

Fig 06: Handling Invalid Input

Description: Passed negative grade value to check if invalid entries are handled correctly.

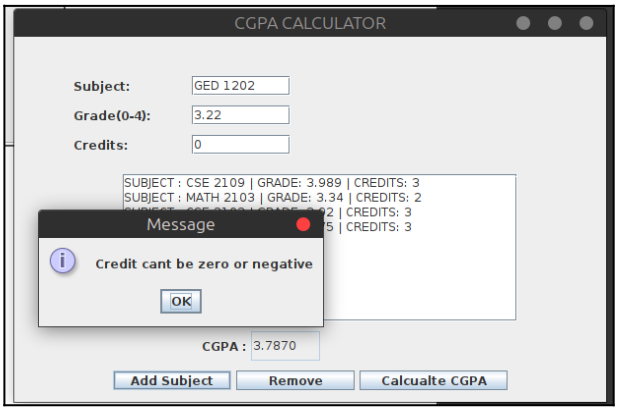

Fig 07: Handling Invalid Input

Description: Passed credits value as zero to check if invalid entries are correctly handled.

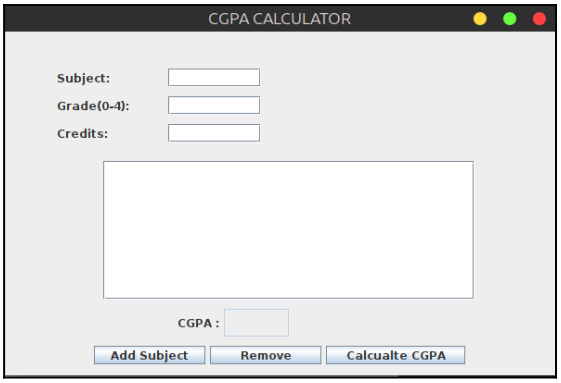

Fig 08: Remove button clears all entries

Description: Pressing “Remove” button the application clears all entries, including text area.

Discussion:

1. Interpretation:

This application ticks all its objectives, by automating Cumulative Grade Point Average calculation. The weighted average formula ensures the correctness of the results which aligns with academic standards. Following Java Swing tutorials, the GUI implementation is pretty straightforward, labels are used to define what to be given as entry, text fields gather input, buttons trigger actions (add, remove,calculate), and a text area lists entered subjects. Lastly, input validation prevents invalid data entries from altering results or crashing the program.

So, by automating the CGPA calculation process, this application saves time and provides accuracy over multiple course's.

2. Challenges:

1. **Input Validation:** Making sure that all inputs are numeric, and are within the valid range (grades), and credits are positive & non-zero.
2. **Parsing:** As text-fields takes String values, they are parsed into numeric values and these formatted texts introduces risk of error.
3. **GUI Layout Management**: Manual positioning of components makes alignment difficult.
4. **Input Handling:** Managing multiple subject entries and ensuring that they are stored correctly while providing accurate final CGPA.
5. **Error Handling:** Providing clear feedback for invalid inputs, so user can know what went wrong.
6. **Accurate Calculation:** Correct implementation of weighted formula and avoiding division errors.

3. Limitations:

1. **Fixed Grading Scale:** The application only supports the 4.0 scale GPA. It cannot calculate CGPA on other scales like 5.0 or 10.0 or percentage without modification.
2. **Data is Temporary:** All data is temporary, as the program is closed all data is lost.
3. **Desktop Only:** The Java Swing interface works on desktop environments only.
4. **Manual Data Entry**: Users have to type each subject's data manually. There is no import option from data sheets.

Conclusion:

1. Summary:

The Cumulative Grade Point Average Calculator application was successfully developed using Java Swing. It achieves all its target, by automating the cumulative grade point average calculation process over multiple subjects, thereby saving time and improving accuracy for students. The GUI (labels, text fields, buttons) and action-events (add, remove, and calculate) work together to meet users needs. Test cases demonstrate that the application provides correct result in all tested scenarios and effectively handles expected invalid inputs.

2. Contributions:

Key contributions include:

1. **Practical Benefits:** This CGPA Calculator significantly reduces manual grade calculation time from several minutes to seconds. It works offline on any Java-enabled computer.
2. **Technical Contributions:** This application demonstrates strict input validation through multiple stages, like empty fields check, numeric parsing; grades and credits range check, while preventing crashes and providing clear error feedback to users.
3. **Educational Value:** The project is a practical demonstration of integrated GUI design, event-driven programming, and exception handling in Java. And well-commented code provides a swift learning curve for new programmers.

3. Future Work:

Potential enhancements for the CGPA calculator includes:

1. **Adding Database:** Adding a database would allow users to save their subjects and CGPA history.
2. **Multi-scale Support:** Implementing such components that will handle other scaling systems like 5.0, 10.0, or percentage-based grading.
3. **User Profiles:** Allowing users to keep their own records instead of sharing one temporary session.
4. **Web or Mobile Version:** Converting current desktop version into an web application or mobile app.
5. **Design Enhance:** Adding a file import option (e.g., CSV).

References:

[1] GeeksforGeeks. (2025d, July 15). *Student Grade Calculator using Java Swing*. GeeksforGeeks. https://www.geeksforgeeks.org/java/student-grade-calculator-using-java-swing/

[2] *Java Swing Bangla Tutorials*. (n.d.). YouTube. https://youtube.com/playlist?list=PLgH5QX0i9K3rAHKr6IteF5kdgN6BorH9l&si=N_Hz3he6ad-TJmEh

[3] *Calculate GPA | Additional resources | Helpful resources | Graduate and Professional Admissions | The Ohio State University*. (n.d.-e). https://gpadmissions.osu.edu/resources/calculate-gpa.html